Programmazione orientata agli oggetti

Fabio Bonsignore

3 Marzo 2025

1 Astrazione e UML

1.1 Classi e istanze

Definizione 1.1.1. *Una classe è un concetto astratto che rispetta delle caratteristiche, l'istanza è più concreta e dettagliata.*

Mentre le classi sono dinamiche, gli oggetti sono statici.

Esempio 1.1.1. *In "Fido è un Cane", Fido è un'istanza mentre Cane è la classe.*

Definizione 1.1.2. *La struttura che le classi vanno a costituire si chiama albero, le istanze sono dette foglie.*

Definizione 1.1.3. *Un'astrazione è un procedimento concettuale per mezzo del quale si definisce un concetto più generale a partire da concetti più elementari, per mezzo della rimozione di dettagli e particolari.*

Le astrazioni strutturali modellano oggetti e le relazioni tra di essi.

1.2 Astrazioni di struttura

Definizione 1.2.1. *La classificazione ("instance of") lega istanze e classi.*

Ogni istanza possiede le caratteristiche della classe di appartenenza.

Definizione 1.2.2. *La generalizzazione ("is a") lega una classe genitore (superclasse) a una o più classi figlie (sottoclassi) che ne sono sottoinsieme.*

Dunque, ogni oggetto della sottoclasse appartiene anche alla superclasse.

Esempio 1.2.1. *In "La mia Automobile è una Cinquecento", Cinquecento è una sottoclasse di Automobile ma l'istanza è la macchina con la targa.*

Definizione 1.2.3. *L'aggregazione ("part of") lega una classe "Aggregato" con un insieme di classi "Parti".*

Ogni oggetto di Aggregato è costituito dagli oggetti delle classi Parti, ma la distruzione di Aggregato non porta a quella delle Parti.

Esempio 1.2.2. *In "Un Cane è parte di un Branco", Cane è una classe Parti e Branco è la classe Aggregato.*

Definizione 1.2.4. *La composizione è un'aggregazione più forte, per la quale la distruzione della classe Aggregato porta a quella delle Parti.*

Definizione 1.2.5. *L'associazione ("has a") definisce una connessione logica fra oggetti di classi.*

Esempio 1.2.3. *In "Una Persona è padrone di un Cane", "è padrone" indica l'associazione tra l'oggetto Persona e Cane.*

1.3 Formalismi di rappresentazione

I formalismi di rappresentazione vedono il Personal Computer come ordinatore. I principali sono ERD (Entity Relationship Diagram) che nasce nell'ambito di Basi Dati e UML (Unified Modeling Language) che nasce nell'ambito di Programmazione. Noi useremo l'UML, che è fondamentale in fase di analisi e progetto.

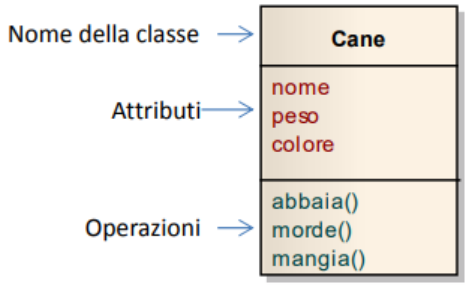


Figure 1: Rappresentazione di una classe

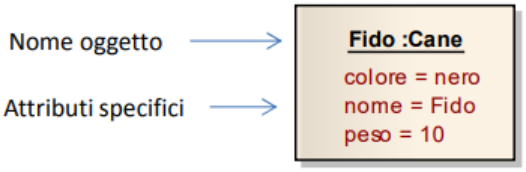


Figure 2: Esempio di oggetto

Vediamo ora le caratteristiche che possono essere associate a una freccia:
- Nome: esprime il significato dell'associazione, solitamente è un verbo.
- Ruolo: esprime i ruoli dei partner, solitamente è un sostantivo o un aggettivo.
- Cardinalità: esprime quante istanze di una classe possono essere associate all'altra. L'$*$ indica molti; altri esempi sono 1..* almeno 1, 2 esattamente 2, 1..5

da 1 a 5, $1, 4, 6$ o 1 o 4 o 6.
- Navigabilità: esplicita chi vede cosa.

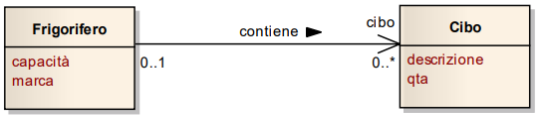


Figure 3: Esempio di associazioni

Inoltre, esistono diversi tipi di associazioni:
- Associazione multipla: quando più linee legano le stesse classi.
- Associazione a cappio: una linea che collega la stessa classe ma con ruoli diversi. C'è il rischio che si crei ambiguità.

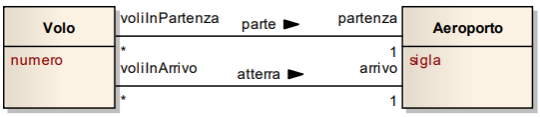


Figure 4: Esempio di associazione multipla (si presume che il numero di volo rimanga lo stesso)

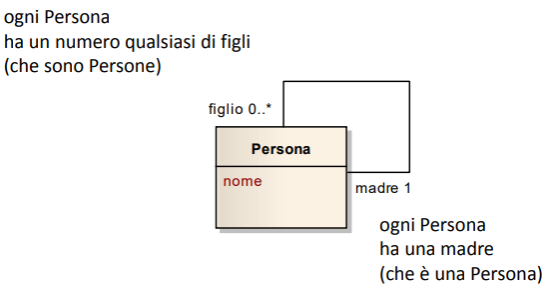


Figure 5: Esempio di associazione a cappio

Infine, vediamo i simboli per aggregazione e composizione.



Figure 6: Linea di aggregazione

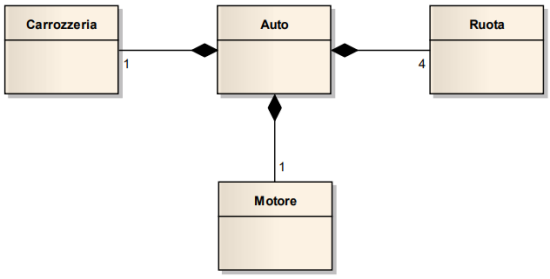


Figure 7: Linea di composizione

1.4 Programmazione orientata agli oggetti in Java

Nel corso di "Laboratorio di matematica e informatica", abbiamo trattato Java come linguaggio imperativo, tuttavia è un linguaggio orientato agli oggetti.

Definizione 1.4.1. *Un oggetto è un insieme di dati e metodi, ciascuno dotato di un'identità che li distingue dagli altri oggetti con lo stesso stato. I dati determinano lo stato dell'oggetto, mentre i metodi realizzano gli algoritmi per manipolare lo stato.*

Gli attributi di un oggetto sono le variabili e possono essere semplici o a loro volta oggetti. Non bisogna aggiungere variabili non specifiche dell'oggetto nei suoi dati (ad esempio, un contatore per un while).
Gli oggetti comunicano scambiandosi messaggi (o comandi) attraverso le interfacce, attivando i metodi che determinano il comportamento.
L'identità è dovuta all'object identifier OID, che è univoco nel sistema e invariante nel tempo per tutta la vita dell'oggetto.

Definizione 1.4.2. *La classe è il modello attraverso il quale rappresento gli oggetti. L'istanza (o oggetto) è un esemplare creato attraverso una classe.*

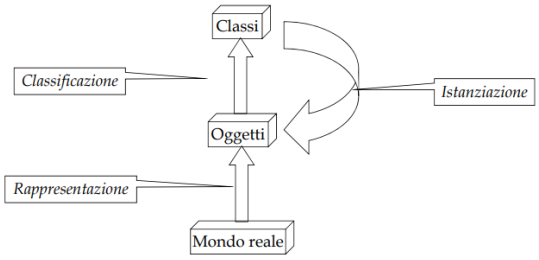


Figure 8: Processo di astrazione

Nella classe, definisco la signature dei messaggi (nome, tipo e parametri formali degli attributi visibili dall'esterno) e implementa attributi e metodi interni e le operazioni dei metodi.

Definizione 1.4.3. *L'information hiding è uno dei principi dell'OOP e si basa sulla divisione in un'interfaccia visibile (comandi invocabili da altri oggetti) e un'implementazione invisibile (variabili locali e metodi che eseguono comandi).*

L'utilità sta nell'impedire all'utente di alterare il funzionamento del programma (ad esempio, non siamo noi che pesiamo il Cane ma è il Cane che contiene l'informazione del peso).

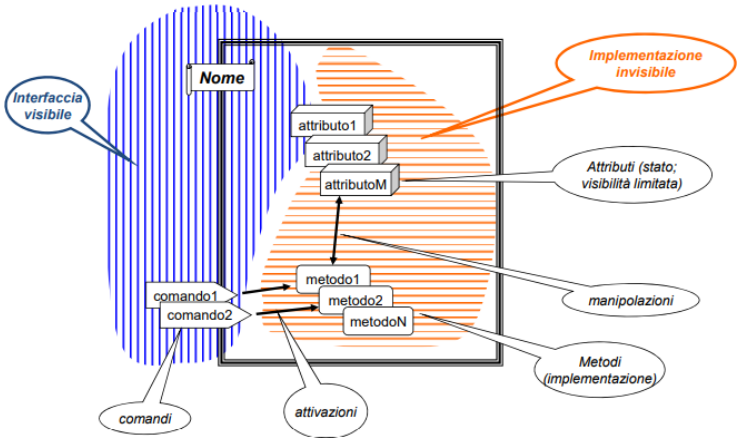


Figure 9: Information Hiding

In Java, una classe si crea come
public [modificatori] class NomeClasse {
Attributi d'istanza (variabili locali)
Costruttori
Metodi
}

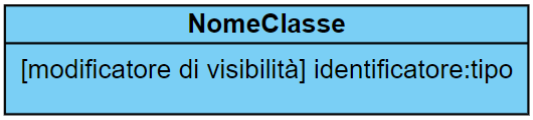


Figure 10: Struttura di classe in UML

Il modificatore di visibilità è "+" se è public, "-" se è private. Un oggetto di una classe si crea come
NomeClasse nomeIstanza; (dichiarazione)
nomeIstanza = new NomeClasse(); (creazione)
Quando assegno un oggetto a un altro, sto facendo aliasing. Questo significa che l'oggetto che viene riassegnato perde la sua identità e punta allo stato puntato dall'oggetto. L'aliasing non è transtitivo, dunque se a = b e b = c, allora a

continuerà a puntare lo stato che era puntato da b, mentre b inizierà a puntare c.

1.5 Gestione della memoria

Definizione 1.5.1. *Il garbage collector libera la memoria non più referenziata.*

A livello macchina, la memoria è organizzata logicamente come un insieme di locazioni accessibili tramite indirizzi. A livello evoluto (Java), la memoria è organizzata in zone con meccanismi di allocazione diversi: static, stack e heap.

Definizione 1.5.2. *Lo static è dove vengono memorizzate le variabili statiche (gli attributi di classe) che hanno vita pari a quella del programma e che vengono allocate all'avvio.*

Definizione 1.5.3. *Lo stack è dove vengono memorizzate le variabili dinamiche che hanno vita pari a quelle del metodo che le ha invocate.*

Definizione 1.5.4. *L'heap è dove vengono memorizzate le variabili dinamiche allocate quando viene chiamato un new e vengono deallocate automaticamente da Java quando non servono più.*

La costante "null" è il valore di default per gli oggetti. Non è possibile usare "equals" se l'oggetto da comparare ha valore null, tuttavia è possibile comparare un oggetto con null.
La keyword "instanceof" permette di controllare se un oggetto sia istanza di una certa classe.

1.6 Associazioni

Abbiamo quindi che un sistema è composto da più classi e il mezzo attraverso cui gli oggetti interagiscono sono le associazioni. Il legame che viene instaurato è di tipo semantico e c'è un link tra gli oggetti.

Definizione 1.6.1. *Un link è un'istanza di associazione.*

Di default ogni associazione è bidirezionale, ma può essere resa unidirezionale specificandone la direzione. Se A 'vede' B, significa che la classe B è tra gli attributi della classe A.
Le associazioni a cappio vengono codificate attraverso le liste. A differenza degli array, la loro dimensione può essere modificata, tuttavia per accedere a un elemento della lista bisogna passare per i precedenti (o i successivi).

Esempio 1.6.1. *Se voglio codificare un treno, creo una classe Vagone che ha al suo interno un oggetto Vagone chiamato next. In questo modo, posso formare il treno come vagone.next(),vagone.next.next(),...*
Similmente, si possono associare all'indietro mettendo ad ogni vagone (tranne al primo) un oggetto Vagone previous.

2 Programmazione in Java

2.1 Metodi

Definizione 2.1.1. *Un metodo è definito da un intestazione (o signature), che permette di invocarlo, e un body, che contiene le istruzioni.*

Si dividono in metodi che eseguono esclusivamente istruzioni e in quelli che restituiscono un valore dopo aver eseguito istruzioni. I primi usano la keyword "void", i secondi usano il tipo ritornato e l'ultima istruzione del body deve essere "return variabile".
Si intestano come "[modificatore] tipoRitorno omettendo([lista parametri formali])"
Tutti i metodi devono essere descritti e la descrizione deve contenere precondizioni e postcondizioni.

Definizione 2.1.2. *Le precondizioni indicano le condizioni che devono essere vere per invocare il metodo, le postcondizioni descrivono gli effetti prodotti dall'invocazione del metodo.*

In Java, si usa il comando "javadoc" per generare un file HTML con la descrizione dell'interfaccia pubblica.

Definizione 2.1.3. *Operare overloading di un metodo significa assegnare lo stesso nome a due definizioni differenti nella classe.*

Si fa quando diversi metodi semanticamente fanno la stessa cosa ma con parametri diversi. Java decide quale chiamare sulla base del numero e del tipo di argomenti. Non basta cambiare il tipo ritornato, devono cambiare anche i parametri formali.

2.2 Incapsulamento

Definizione 2.2.1. *L'incapsulamento permette di separare l'interfaccia (ciò che è visibile dall'esterno) dall'implementazione (i dettagli che si vogliono nascondere agli user).*

L'idea fondamentale dietro è che il programmatore deve conoscere solo cosa fa il programma e non come viene fatto. Il vantaggio principale è che permette di modificare l'implementazione senza che le classi che la utilizzano se ne accorgano. Tutto quello che dichiaro public appartiene all'interfaccia.

Definizione 2.2.2. *Un metodo di incapsulamento è un metodo che permette a un'altra classe di leggere o impostare il valore di un attributo di istanza dichiarato private.*

Questi metodi devono appartenere all'interfaccia pubblica della classe. Servirà 1 metodo per dotare la classe per accedere o solo in lettura o solo in scrittura, 2 per permettere entrambi. La convenzione è per il primo "getNomeAttributo",

per il secondo "setNomeAttributo".
La keyword "this" permette di riferirsi all'attributo di istanza. In sintesi:
(1) Bisogna inserire un commento in testa che specifica ciò che rappresenta.
(2) Dichiarare gli attributi di istanza all'implementazione col modificatore "private".
(3) Fornire gli opportuni metodi di incapsulamento col modificatore "public".
(4) Fornire i metodi dell'interfaccia col modificatore "public".
(5) Documentare tutti i metodi con precondizioni e postcondizioni.
(6) Dichiarare i metodi di servizio appartenenti all'implementazione col modificatore "private".
(7) Aggiungere sempre nella definizione di classe il metodo toString().

2.3 Costruttori

Definizione 2.3.1. *Un costruttore è un particolare metodo che viene invocato solo alla creazione dell'oggetto.*

La sua funzione è quella di inizializzare l'oggetto direttamente quando viene creato. Di default, il costruttore di Java inizializza ogni attributo di un oggetto a un particolare valore (null per gli oggetti, 0 per gli int, ...). Sono definiti in questo modo:

public NomeOggetto([parametri formali]) {
[Inizializzazioni]
}

Nel momento in cui viene definito un costruttore, quello di default viene cancellato automaticamente. E' quasi sempre conveniente fare overloading con i costruttori, definendone uno generale con tutti gli attributi e definendo gli altri in funzione del primo.

Definizione 2.3.2. *La keyword "this" permette di specificare che si sta facendo riferimento all'oggetto.*

Esempio 2.3.1. *Se ho un setter che imposta a un attributo nome l'input nome, devo scrivere "this.nome = nome;" per distinguere i due nome.*

Per invocare il costruttore facendo overloading come descritto prima, si scrive "this([parametr formali]).

3 Ereditarietà

3.1 Concetti base

Definizione 3.1.1. *L'ereditarietà è una tecnica attraverso la quale posso definire una classe più generale alla quale legare delle sottoclassi, che possiedono le caratteristiche della classe generale più delle caratteristiche specifiche.*

Dunque, ogni sottoclasse ha tutte le caratteristiche della superclasse. In UML, si rappresentano con una freccia dalla sottoclasse alla superclasse. Se la generalizzazione è il legame tra le classi e le superclassi, la specializzazione è il legame opposto rappresentato dall'ereditarietà.
Essendo le sottoclassi create a partire dalle superclassi, è corretto usare la keyword per capire se l'oggetto della sottoclasse appartiene anche alla superclasse. Per questo, è lecito scrivere "Superclasse ref = new Sottoclasse()" ma non "Sottoclasse ref = new Superclasse()".

Definizione 3.1.2. *La keyword "extends" nella definizione della classe permette di applicare la tecnica di ereditarietà, usando la sintassi "public class NomeSottoclasse extends NomeSuperclass".*

Come tecnica, dispone di diversi vantaggi:
- Economia: basta che definisco una volta le caratteristiche comuni.
- Consistenza: se cambio qualcosa nella superclasse, viene cambiato automaticamente nelle sottoclassi.
- Riuso: posso riusare la superclasse in contesti diversi.
- Estendibilità: posso sempre definire nuove sottoclassi e ogni sottoclasse può fare overriding modificando gli attributi e operazioni.

3.2 Visibilità

Per quanto riguarda la visibilità, una sottoclasse eredita tutto a prescindere dalla visibilità, tuttavia un attributo private nella superclasse non è accessibile tramite nome ma servono metodi. Similmente, se un metodo è private nella superclasse non potrà essere invocato dalle sottoclassi.

Definizione 3.2.1. *La keyword "protected" ("#" in UML) è un attributo di visibilità che permette alle sottoclassi di accedere direttamente ad attributi e metodi definiti nella superclasse.*

3.3 Costruttori e sottoclassi

Per quanto riguarda i costruttori, ogni classe deve inizializzare i propri attributi. Quindi, un costruttore della sottoclasse deve inizializzare per primi i valori della superclasse invocandone il costruttore tramite la keyword "super", dopodiché si inizializzano gli attributi della sottoclasse. Se la classe genitore ha il costruttore di default, allora posso non invocare il "super" nel costruttore della classe figlia se voglio che sia usato quello di default.

3.4 Classe Object

In Java esiste una classe di default "Object", e tutte le classi che vengono definite ne sono una specializzazione. In Object, sono definiti metodi importanti come "equals" e "toString"

3.5 Cast

Seppure sia lecito a livello di compilazione, quello che viene creato con l'istruzione "Superclasse ref = new Sottoclasse()" è comunque un oggetto della superclasse. Perciò, se una sottoclasse inizializzata in questo modo ha un metodo non presente nella superclasse darà errore la chiamata; se invece è presente nella superclasse viene fatto un override.
Se ho un oggetto inizializzato con quell'istruzione, non posso fare "Sottoclasse sub = ref", in quanto ref risulta essere di tipo Superclasse. Posso tuttavia forzarlo facendo casting, dunque "Sottoclasse sub = (Sottoclasse) ref".
Col comando "ref instanceof Classe" posso verificare se un oggetto è un'istanza di una classe.

4 Classi Astratte

4.1 Creazione

Definizione 4.1.1. *Una classe astratta è una classe che viene inizializzata solo allo scopo di essere una superclasse.*

Non esistono dunque oggetti di tipo astratto, ma solo sue sottoclassi. Una classe deve essere definita astratta se ha almeno un metodo astratto, ma può essere definita astratta anche se non ne ha nemmeno uno. In UML, il loro nome è in corsivo.
In Java, le classi astratte si inizializzano come "public abstract classe NomeClasse()" e i metodi come "[visibilità] abstract void nomeMetodo([parametri]);".

4.2 Ereditarietà

Se una sottoclasse estende una classe astratta ma non implementa il body di un metodo abstract, allora è anch'essa abstract.

4.3 Interface

In Java, non esiste l'ereditarietà multipla (ovvero una sottoclasse che eredita da più superclassi), ma può essere realizzata con le classi interface. A differenza delle classi astratte,